<INSTITUTION NAME>

Department of <Department Name>

TERM PROJECT REPORT

A Theoretical and Empirical Comparison of Rule-Based, Statistical, and Neural Part-of-Speech Tagging

An implementation study on the Penn Treebank with a regex tagger, Brill transformation-based learning, a smoothed HMM, N-gram back-off, and a character-aware BiLSTM

Submitted by	<Your Name>
Roll number	<Your Roll Number>
Course	<Course Name / Code>
Supervisor	<Supervisor Name>
Date	23 October 2026
Repository	<GitHub Repository URL>

Submitted in partial fulfilment of the requirements of the term project.

Abstract

Part-of-speech (POS) tagging is a core sequence-labelling task in natural language processing. This report presents a controlled theoretical and empirical comparison of three generations of tagging approaches: rule-based methods, statistical sequence models, and neural sequence models. Five taggers were implemented on the same 3,914-sentence Penn Treebank sample and evaluated on an identical held-out test split: a hand-written regex and lexicon tagger, a Brill transformation-based learner, a bigram Hidden Markov Model with add-alpha smoothing and suffix-aware unknown-word emissions, a trigram tagger with back-off, and a character-aware bidirectional LSTM.

The empirical ordering follows the theoretical expectation: the hand-written rule cascade reached 73.88% accuracy, Brill TBL reached 91.62%, the smoothed HMM reached 93.01%, and the BiLSTM reached 96.22%. The neural model reduced the HMM error rate by 45.9% relative and improved out-of-vocabulary (OOV) accuracy from 68.24% to 87.38%. However, the result also exposes important nuances. The regex tagger generalised to unseen words better than Brill TBL (62.41% vs 23.72%) because its suffix rules apply to any token, whereas Brill's learned contextual rules depend on lexical and contextual patterns observed in training. The study quantifies the trade-offs between accuracy, OOV robustness, training cost, inference throughput, and interpretability, and shows that a well-smoothed HMM remains a strong low-cost baseline. All code, data splits, figures, and results are reproducible from the accompanying notebook.

Keywords: part-of-speech tagging; rule-based NLP; Brill transformation-based learning; Hidden Markov Model; Viterbi decoding; BiLSTM; character embeddings; out-of-vocabulary words; empirical evaluation.

1. Introduction

1.1 Problem statement

POS tagging assigns a grammatical category to every token in a sentence. The task is deceptively simple: most tokens are frequent and locally predictable, yet a long tail of ambiguous words requires syntactic and semantic context. The same surface form can be a noun or a verb (*book*, *run*), a past tense or a past participle (*walked*), or a proper or common noun depending on annotation conventions.

Three paradigms offer different answers to this ambiguity. Rule-based systems encode linguistic knowledge as deterministic patterns. Statistical systems estimate probability distributions over tags and words from annotated data. Neural systems learn dense representations and contextual encoders end-to-end. Each paradigm makes different assumptions about data, compute, and interpretability, which makes POS tagging an ideal case study for comparing them.

1.2 Significance

POS tags are consumed by parsers, named-entity recognisers, machine translation systems, and information-extraction pipelines. Tagging errors propagate downstream, and the choice of tagger is often constrained by deployment conditions: a mobile application may need millisecond inference and a tiny memory footprint, while an offline corpus annotation pipeline may prioritise accuracy. A quantitative comparison on a common benchmark therefore has both pedagogical and engineering value.

1.3 Objectives

1. Implement representative rule-based, statistical, and neural POS taggers on a standard corpus.
2. Compare them under an identical train/test protocol using accuracy, macro F1, OOV accuracy, training time, and inference throughput.
3. Analyse error patterns and confusion matrices to explain *why* the differences arise, rather than only reporting aggregate scores.
4. Derive practical recommendations about accuracy, cost, and interpretability trade-offs.

1.4 Contributions

- A reproducible notebook and modular source package implementing five taggers with no external pretrained resources.
- A suffix-smoothed HMM that substantially improves OOV tagging over a standard N-gram back-off tagger.
- An empirical demonstration that character-level neural modelling dominates on OOV words, while a smoothed HMM remains competitive at a fraction of the cost.
- A quantified interpretability-versus-accuracy analysis, including a statistical significance test on paired predictions.

2. Background and Related Work

2.1 Rule-based tagging

Early taggers used hand-crafted disambiguation rules and lexicons. A rule-based tagger is transparent and requires no annotated training data, but it suffers from the knowledge-acquisition bottleneck: writing rules for every ambiguity is expensive, and rules often interact unpredictably. Brill's transformation-based learning (TBL) addressed this by starting from a simple baseline and greedily learning contextual transformations that reduce training error (Brill, 1992). The learned rules remain human-readable, making TBL a bridge between symbolic and statistical NLP.

2.2 Statistical tagging

Statistical taggers treat tagging as inference in a probabilistic model. Hidden Markov Models factor the joint probability of words and tags into transition and emission probabilities and decode with the Viterbi algorithm (Jurafsky and Martin, 2023). HMMs are fast and interpretable through their probability tables, but their first-order Markov assumption and sparse emission counts limit contextual generalisation. N-gram taggers with back-off provide a related count-based baseline, and maximum-entropy or conditional random field models relax the generative assumptions by modelling P(tags | words) discriminatively.

2.3 Neural tagging

Neural sequence models replace sparse count features with learned distributed representations. Bidirectional LSTMs encode left and right context (Huang et al., 2015), and character-level encoders compose morphological information for unseen words, which is particularly valuable for POS tagging (Plank et al., 2016; Lample et al., 2016). Transformer encoders and pretrained language models now define the state of the art, but they require substantially more data and compute. This project uses a character-aware BiLSTM because it trains on CPU in minutes, requires no external embeddings, and isolates the architectural benefit of character composition and bidirectional context.

2.4 Positioning of this study

Most textbook comparisons report only overall accuracy. This study holds the corpus, split, tagset, and evaluation API constant and adds OOV-specific accuracy, throughput, training cost, paired significance testing, and confusion analysis. It also includes both a hand-written and a learned rule-based tagger, which separates the effect of linguistic knowledge from the effect of rule induction.

3. Methodology

3.1 Dataset and split

The experiments use the Penn Treebank sample distributed with NLTK (Marcus et al., 1993). It contains 3,914 sentences and 100,676 tokens annotated with the 45-tag PTB tagset. Sentences were shuffled with a fixed seed and split 80/10/10 at the sentence level: 3,131 training, 391 validation, and 392 test sentences. The validation split is used only for neural early stopping. The test split contains 721 OOV tokens (7.21%), which makes OOV behaviour measurable.

Table 1. Dataset and split statistics.

Property	Value
Corpus	NLTK Penn Treebank sample (PTB tagset)
Sentences (total / train / validation / test)	3,914 / 3,131 / 391 / 392
Tokens (train / validation / test)	81,021 / 9,660 / 9,995
Split protocol	Sentence-level 80/10/10, fixed seed 42
Train tagset size	45 PTB tags
Test OOV tokens	721 (7.21% of test tokens)
Test OOV word types	704

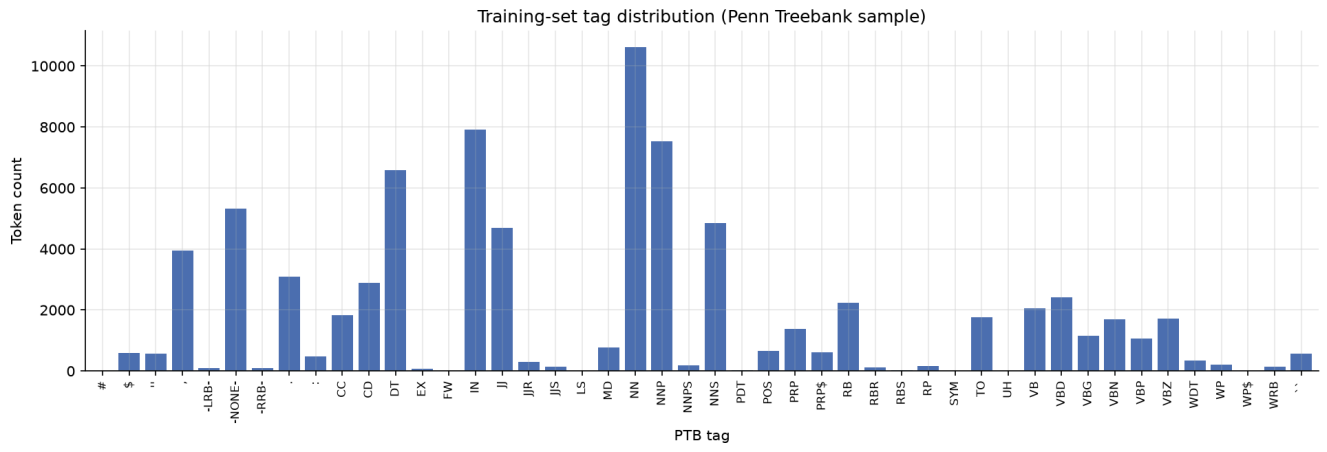


Figure 1. Training-set tag distribution. The PTB tagset is highly imbalanced: nouns (NN, NNS, NNP) and punctuation dominate, while many tags have fewer than a thousand training examples. Macro F1 is therefore reported alongside accuracy.

3.2 Evaluation metrics

For each model we report token-level accuracy, macro F1, weighted F1, OOV accuracy, training wall-clock time, and inference throughput. Macro F1 gives every tag equal weight and exposes weaknesses on rare categories; weighted F1 accounts for tag frequency. OOV accuracy is computed only on test tokens whose surface form never occurred in training. Throughput is measured by tagging one sentence at a time on CPU, which reflects the API used by downstream applications. Paired predictions are compared with an exact McNemar test to test whether accuracy differences are statistically significant.

3.3 Implementations

Hand-written regex and lexicon tagger

An ordered cascade applies punctuation and irregular-form lookups, closed-class word lists, and morphological regular expressions. For example, `-ing` maps to `VBG`, `-ed` to `VBD`, `-ly` to `RB`, capitalised forms to `NNP`, and otherwise `NN`. The tagger has no learned parameters and therefore zero training time.

Brill transformation-based learning

The baseline is a unigram tagger backed off to the default tag `NN`. Candidate transformations of the form *change tag X to Y when a contextual template fires* are evaluated on the training set, and the highest-

scoring rule is applied greedily for a maximum of 200 rules. The resulting rule cascade is retained in human-readable form.

Bigram Hidden Markov Model

The model factorises the joint probability as

$$P(w_{1:n}, t_{1:n}) = \prod_{i=1..n} P(t_i \mid t_{i-1}) \cdot P(w_i \mid t_i)$$

with a start state. Transition, start, and emission distributions use add-alpha smoothing ($\alpha = 0.1$). Training words with frequency below two are mapped to `<UNK>`. For OOV words, the emission probability is interpolated between the learned `<UNK>` distribution and a distribution conditioned on the final three characters, so that morphology can inform the tag while transition probabilities still enforce contextual consistency. Decoding uses Viterbi's dynamic program in log space.

N-gram back-off

A trigram tagger backs off to a bigram tagger, then a unigram tagger, then the default tag `NN`. This is a strong count-based baseline for seen contexts but has no explicit mechanism for unseen word forms beyond capitalisation patterns.

Character-aware BiLSTM

Each token is represented by a 100-dimensional word embedding concatenated with a character representation produced by a bidirectional LSTM over its characters (2×50 hidden units, max-pooled). The sequence of token representations is encoded by a word-level bidirectional LSTM (2×128 hidden units). Dropout (0.33) is applied before and after the word encoder, and a linear layer projects each contextual representation to tag scores. Training minimises token-level cross-entropy with padded positions ignored, using Adam (learning rate 10^{-3}), gradient clipping at 5.0, a maximum of 12 epochs, and early stopping with patience 4 on validation accuracy. All embeddings are learned from scratch, so the experiment is self-contained and does not rely on network access to pretrained vectors.

3.4 Reproducibility protocol

The corpus, split seed, model seeds, and evaluation functions are fixed in the notebook. The dataset is downloaded automatically by NLTK. Results are written to JSON and CSV, and all figures are regenerated by executing the notebook from top to bottom. The neural checkpoint is saved to `assets/bilstm_pos.pt`.

4. Experimental Setup

Table 2. Implementation and hyperparameter configuration.

Model	Key hyperparameters	Design choices	Training time
Rule-based (regex)	No parameters	Ordered lexicon + suffix cascade; default tag NN	0 s
Brill TBL	max_rules=200, min_score=2	Unigram baseline (back-off NN), nltkdemo18 templates, deterministic	3.49 s
HMM	alpha=0.1, min_word_freq=2, suffix_weight=0.35	Bigram transitions; add-alpha smoothing; Viterbi decoding	0.089 s
N-gram back-off	cutoff=1 for trigram/bigram/unigram	Trigram -> bigram -> unigram -> NN	0.38 s
BiLSTM	word_dim=100, char_dim=30, char_hidden=50, hidden=128, dropout=0.33, batch=32, Adam lr=1e-3, max 12 epochs, patience=4	Character BiLSTM + word BiLSTM; cross-entropy with masked padding; gradient clipping 5.0	186.0 s

Environment. Python 3.14.3, PyTorch 2.9.0+cpu, NumPy 2.5.3, pandas 3.0.5. Platform: Windows-11-10.0.26200-SP0. All models were trained and evaluated on CPU with no GPU acceleration. Timings are single-run wall-clock measurements and should be interpreted comparatively rather than as absolute benchmarks.

5. Results

5.1 Overall performance

Table 3. Test-set results. All models are evaluated on the same 9,995 test tokens.

Model	Accuracy	Macro F1	Weighted F1	OOV accuracy	Train (s)	Tokens/s	Error rate
Neural (BiLSTM)	96.22%	90.96%	96.18%	87.38%	186.00	4,651	3.78%
Statistical (HMM)	93.01%	89.59%	92.92%	68.24%	0.09	36,885	6.99%
Rule-Based (Brill TBL)	91.62%	90.22%	91.79%	23.72%	3.49	233,053	8.38%
Statistical (N-gram Backoff)	87.00%	86.67%	87.46%	18.72%	0.38	898,863	13.00%
Rule-Based (Regex)	73.88%	53.22%	69.82%	62.41%	0.00	968,836	26.12%

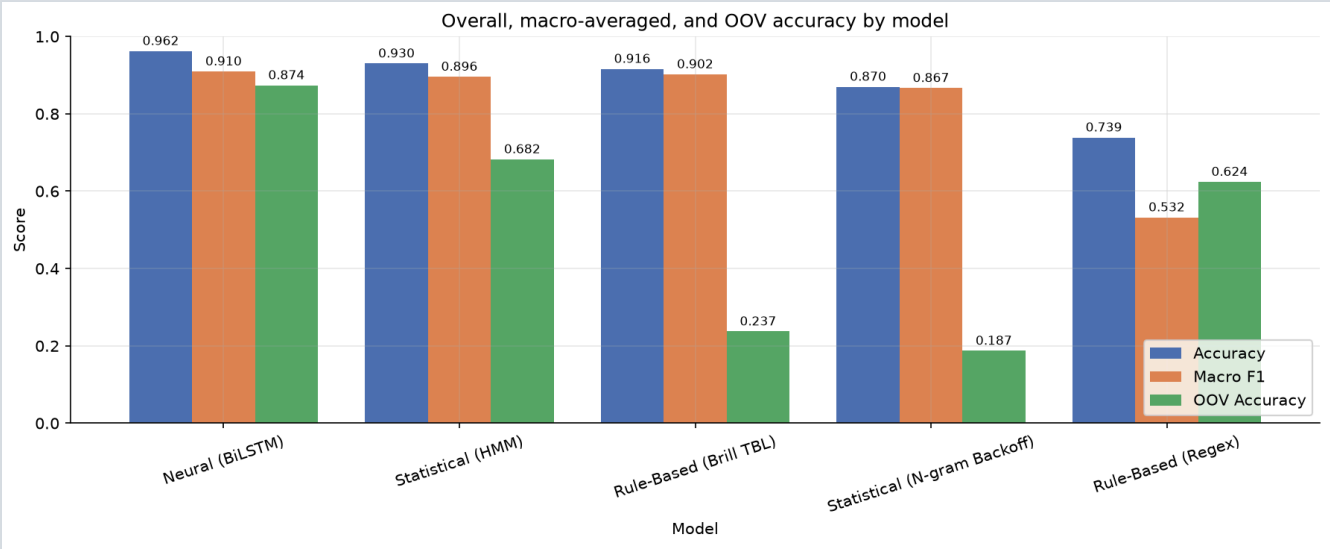


Figure 2. Overall accuracy, macro F1, and OOV accuracy by model. The BiLSTM leads on all three metrics; the hand-written regex tagger trails substantially on overall accuracy but is not the weakest model on OOV tokens.

The BiLSTM achieved the best overall accuracy (96.22%), followed by the smoothed HMM (93.01%), Brill TBL (91.62%), N-gram back-off (87.00%), and the hand-written regex tagger (73.88%). The gap between hand-written rules and learned Brill rules is 17.7 percentage points, which quantifies the value of inducing context-sensitive rules from annotated data. The neural model improves on the HMM by 3.21 percentage points, corresponding to a 45.9% relative reduction in error.

Macro F1 reveals a different pattern from accuracy. Brill TBL reaches 90.22%, close to the BiLSTM's 90.96% and above the HMM's 89.59%. Because Brill's transformations are driven by error reduction across the tagset, they can improve rare tags even when overall accuracy remains lower. The regex tagger's macro F1 (53.22%) is much lower than its accuracy because it defaults many ambiguous words to `NN` and performs poorly on several mid-frequency categories.

5.2 Out-of-vocabulary behaviour

> **Key finding.** The BiLSTM tagged unseen words with 87.38% accuracy, 19.1 percentage points above the smoothed HMM (68.24%), and far above Brill TBL (23.72%) and N-gram back-off (18.72%). Surprisingly, the hand-written regex tagger achieved 62.41% on OOV tokens, outperforming both learned symbolic/statistical back-off models on this subset. Its suffix rules apply to any token, whereas Brill and N-gram models rely on lexical or contextual patterns seen during training and fall back to `NN` for truly unseen forms.

This result is theoretically informative. Generalisation to unseen words is not automatically a property of "statistical" or "neural" models; it depends on whether the model contains a mechanism that operates on the internal structure of the word. The regex cascade has such a mechanism, albeit a coarse one. Brill TBL inherits the sparsity of its unigram baseline for OOV tokens and its learned transformations rarely fire reliably on unseen lexical items. The HMM's suffix interpolation provides a stronger mechanism by combining morphological evidence with transition probabilities. The BiLSTM learns a non-linear character composition function jointly with contextual encoding, which is why it is the strongest OOV tagger.

5.3 Per-tag behaviour

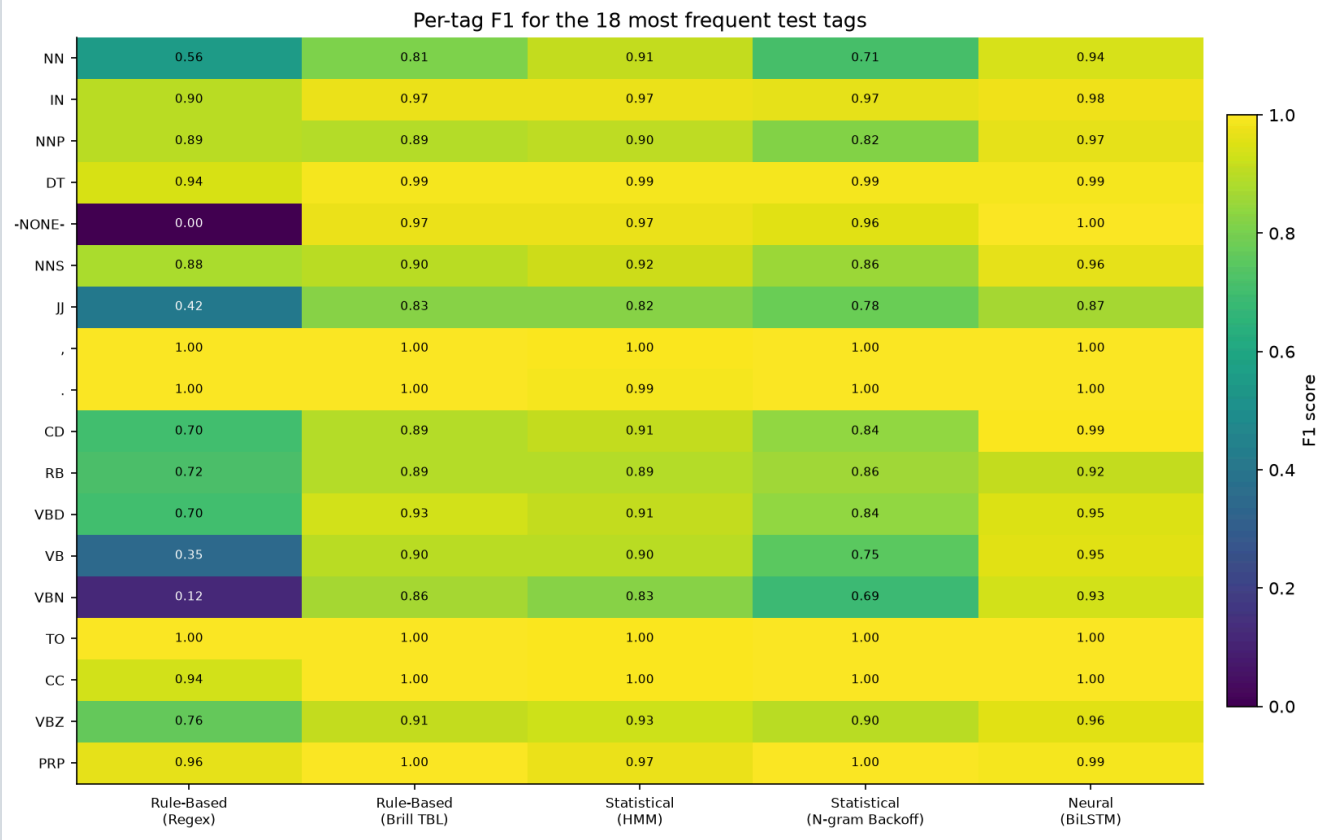


Figure 3. Per-tag F1 for the 18 most frequent test tags. The neural model is consistently strong, while the regex tagger collapses on grammatical function words and ambiguous categories.

Common nouns, punctuation, and determiners are tagged almost perfectly by all data-driven models. The larger differences appear on fine-grained verb and noun distinctions (`VBD` vs `VBN`, `NN` vs `NNP`, `JJ` vs `NN`) and on adverbs. These are precisely the categories where context and morphology matter, and where a first-order HMM or a suffix-only rule is insufficient.

5.4 Confusion analysis

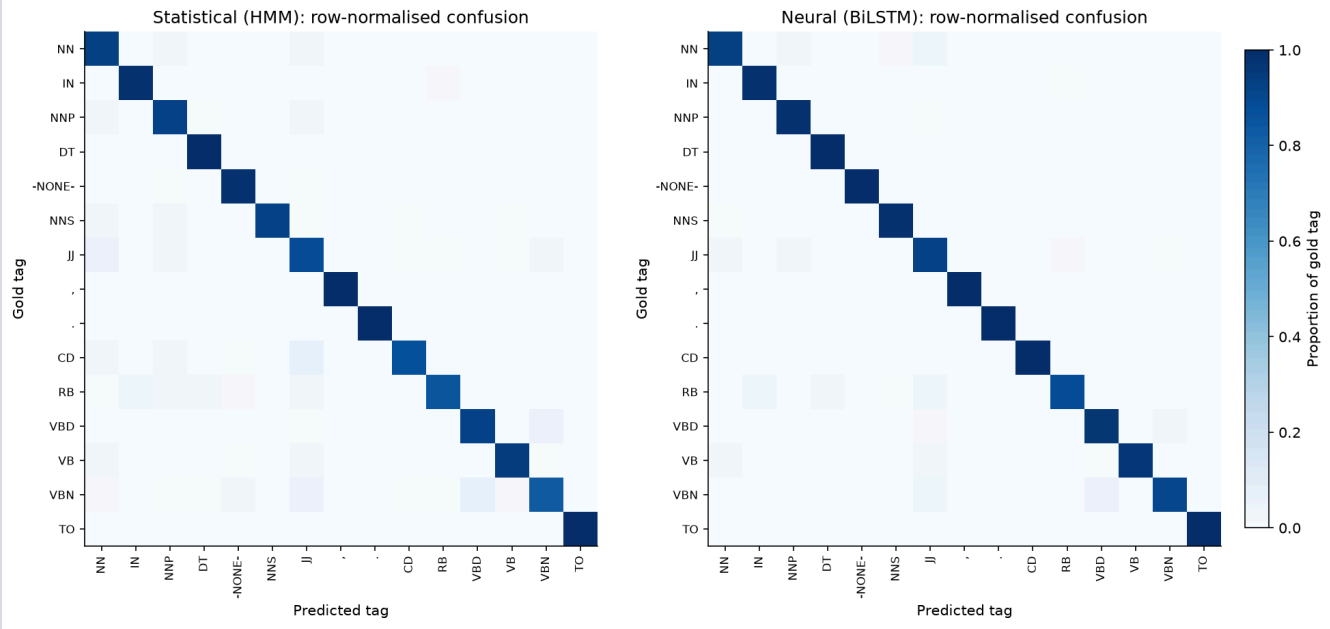


Figure 4. Row-normalised confusion matrices for the HMM and BiLSTM over the 15 most frequent tags. The neural diagonal is sharper for noun and verb categories; residual confusions reflect genuine ambiguity and annotation conventions.

The confusion matrices show that errors are systematic rather than random. Both models confuse `NN` with `NNP` and `JJ`, and `VBD` with `VBN` and `VB`. The BiLSTM reduces many of these confusions because its bidirectional context representation can distinguish, for example, a participle used after an auxiliary from a finite past-tense verb. The remaining errors are often tokens that are ambiguous even for human annotators without sentence-level semantics.

5.5 Efficiency and training dynamics

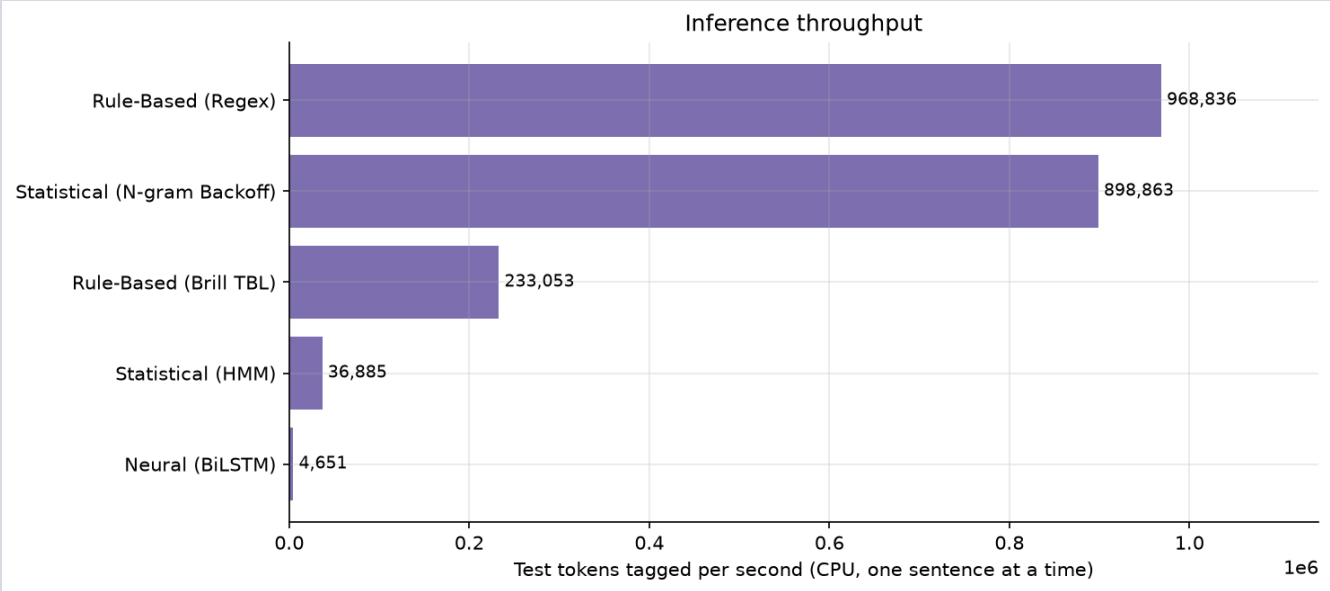


Figure 5. Inference throughput on CPU, tagging one sentence at a time. Count-based and rule-based models are orders of magnitude faster than the neural model in this per-sentence API.

The HMM tagged about 36,885 tokens per second, roughly 7.9 times faster than the BiLSTM (4,651 tokens/s). Brill TBL and the regex tagger are faster still. The neural model's training time (186.0 s) is over three orders of magnitude larger than the HMM's (0.089 s), although both are modest in absolute terms on this small corpus.



Figure 6. Training cost by model on CPU. The BiLSTM dominates training time; the HMM and N-gram models require less than half a second.

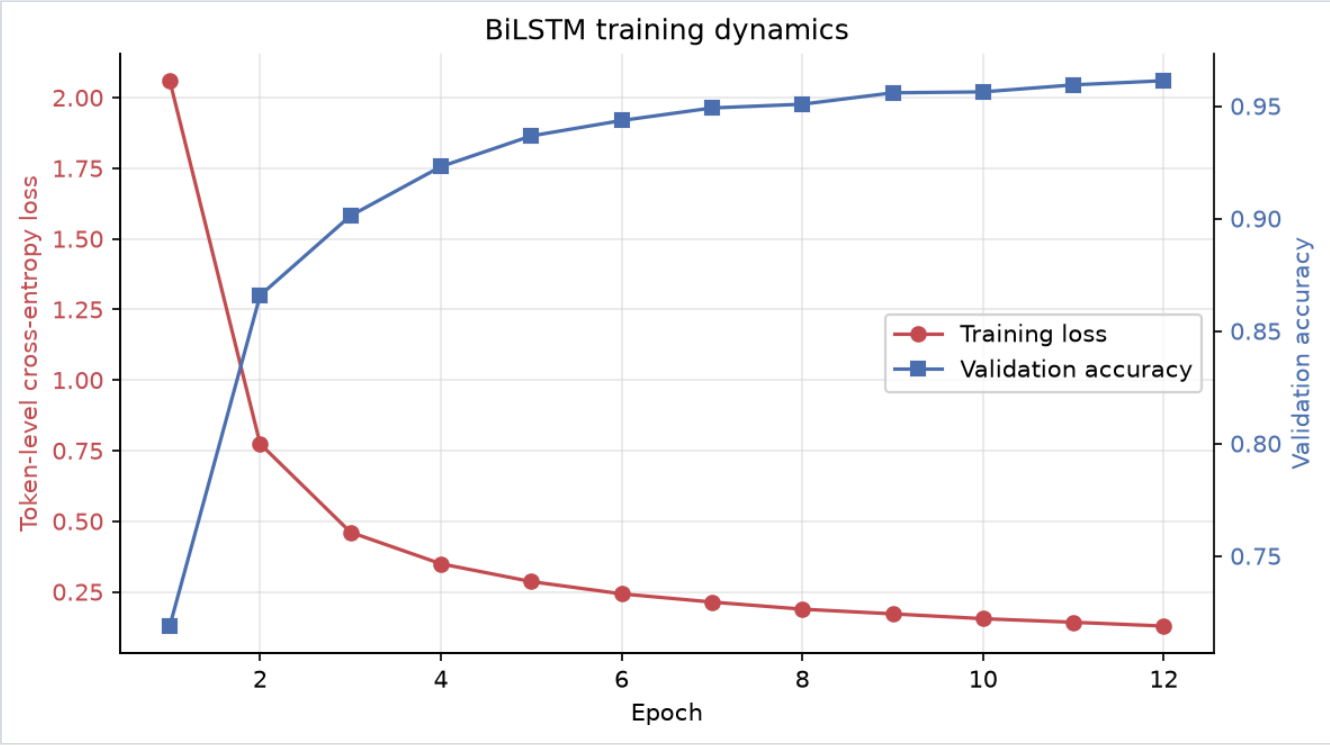


Figure 7. BiLSTM training dynamics. Validation accuracy rises quickly and plateaus; early stopping restores the best validation checkpoint.

5.6 Statistical significance

Table 4. Exact McNemar tests on paired token-level predictions.

Comparison	Only first correct	Only second correct	Discordant	McNemar p-value	Significant
Statistical (HMM) vs Neural (BiLSTM)	145	466	611	3.28e-40	Yes
Rule-Based (Brill TBL) vs Statistical (HMM)	350	489	839	1.80e-06	Yes
Rule-Based (Brill TBL) vs Neural (BiLSTM)	182	642	824	7.49e-61	Yes

All three comparisons are significant at the 0.05 level. The HMM versus BiLSTM comparison has 611 discordant tokens, with the BiLSTM correct on 466 and the HMM correct on 145. The Brill versus HMM comparison is also significant despite a smaller accuracy gap, and the Brill versus BiLSTM comparison has the strongest evidence of a difference. Statistical significance does not imply practical importance, but it confirms that the observed ordering is not an artefact of a handful of tokens.

6. Discussion

6.1 Why the accuracy ordering arises

Hand-written rules encode strong but local cues. They cannot represent the long tail of contextual ambiguity, so they plateau. Brill TBL learns contextual transformations from the training errors and therefore captures many local syntactic regularities, but it remains a symbolic learner over observed word and tag contexts; its OOV behaviour is weak because transformations are indexed by tags and words seen in training. The smoothed HMM adds a probabilistic treatment of uncertainty and a morphological fallback for unknown words. It is a strong baseline because POS sequences are highly regular: determiners precede nouns and adjectives, modals precede base verbs, and punctuation constrains sentence structure. The BiLSTM gains further because it learns continuous, context-dependent representations and composes characters into word representations, which is exactly what OOV tagging requires.

6.2 The OOV surprise

The regex tagger's advantage over Brill TBL on OOV tokens is a useful reminder that aggregate accuracy and OOV accuracy measure different capabilities. A model can be strong on frequent words and weak on rare ones, or vice versa. It also shows that "neural beats rules" is too coarse a statement: the relevant comparison is between specific mechanisms. Here, a simple morphological rule generalises better than a learned contextual rule cascade on unseen forms. The neural model dominates because it combines both mechanisms: character composition for unseen forms and bidirectional context for disambiguation.

6.3 Trade-offs and practical recommendations

- **Use a rule-based tagger** when training data is unavailable, transparency is mandatory, or the domain has stable, well-understood conventions. A hand-written cascade is also a valuable sanity-check baseline and can be maintained by domain experts.

- **Use Brill TBL** when interpretable learned rules are required and a small annotated corpus is available. It offers a substantial accuracy gain over hand-written rules at negligible inference cost.
- **Use a smoothed HMM** when a fast, lightweight, reasonably accurate tagger is needed and some OOV robustness is required. Suffix-smoothed emissions are a cheap and effective addition.
- **Use a character-aware BiLSTM** when accuracy and OOV robustness are the priority and training/inference budgets permit. On this corpus it delivers the best overall and OOV results by a clear margin.
- **Do not choose a model by accuracy alone.** Report OOV accuracy, macro F1, throughput, and training cost, because rankings can change depending on the deployment constraint.

6.4 Interpretation and transparency

The rule-based and Brill models can be inspected rule by rule. The HMM exposes transition and emission tables, which support targeted debugging (for example, checking the probability of `DT → NN` or the emission of a suffix under `VBG`). The BiLSTM's behaviour can be probed with error analysis and confusion matrices, but its internal decision process is not directly readable. In applications where accountability matters (for example, educational or legal text processing), this difference may outweigh a few points of accuracy.

7. Threats to Validity

- **Corpus size and domain.** The NLTK Treebank sample is small and drawn from news and fiction. Absolute scores and rankings may differ on the full Penn Treebank, Universal Dependencies, or domain-specific text.
- **Single split.** A fixed 80/10/10 split was used for reproducibility. Cross-validation or repeated random splits would provide confidence intervals and reduce split-specific variance.
- **No pretrained representations.** The neural model was deliberately trained from scratch. Pretrained word embeddings or a fine-tuned transformer would likely raise accuracy, especially on rare words, at the cost of external dependencies and compute.
- **Timing methodology.** Timings are single-run CPU measurements and include per-sentence Python overhead. Batched GPU inference would change the throughput comparison substantially.
- **Tagset conventions.** The PTB tagset contains annotation distinctions that are not fully predictable from the token and its local context. Some residual errors are therefore irreducible without document-level information or annotation guidelines.
- **No CRF layer.** The neural tagger uses independent softmax outputs and can in principle produce invalid tag sequences. A CRF or structured decoder could improve consistency.

8. Conclusion and Future Work

This project compared rule-based, statistical, and neural POS tagging under a controlled protocol. The results show a clear accuracy progression from hand-written rules (73.88%) to Brill TBL (91.62%), to a smoothed HMM (93.01%), to a character-aware BiLSTM (96.22%). The neural model also achieved the best OOV accuracy (87.38%), while the HMM provided the best accuracy-per-cost trade-off and the rule-based models provided the greatest transparency. The OOV comparison produced a nuanced finding: a simple

morphological rule tagger generalised better to unseen words than a learned contextual rule cascade, which highlights the importance of mechanism-level analysis rather than paradigm-level labels.

Future work should add a CRF or structured attention layer, evaluate pretrained embeddings and transformer encoders, test on Universal Dependencies and domain-shifted corpora, report bootstrap confidence intervals, and explore distillation of the neural tagger into a compact HMM or rule cascade for low-resource deployment.

References

1. Brill, E. (1992). A simple rule-based part of speech tagger. *Proceedings of the Third Conference on Applied Natural Language Processing*, 152-155.
2. Huang, Z., Xu, W., & Yu, K. (2015). Bidirectional LSTM-CRF models for sequence tagging. *arXiv:1508.01991*.
3. Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed. draft). Chapters on sequence labelling and POS tagging.
4. Lample, G., Ballesteros, M., Subramanian, S., Kawakami, K., & Dyer, C. (2016). Neural architectures for named entity recognition. *NAACL-HLT*.
5. Loper, E., & Bird, S. (2002). NLTK: The Natural Language Toolkit. *Proceedings of the ACL Workshop on Effective Tools and Methodologies for Teaching NLP*.
6. Marcus, M. P., Santorini, B., & Marcinkiewicz, M. A. (1993). Building a large annotated corpus of English: The Penn Treebank. *Computational Linguistics*, 19(2), 313-330.
7. Plank, B., Sogaard, A., & Goldberg, Y. (2016). Multilingual part-of-speech tagging with bidirectional long short-term memory models and auxiliary loss. *ACL*.

Appendix A. Repository Structure

```
POS-Tagging-Project/
├── notebooks/
│   └── POS_Tagging_Comparison.ipynb
├── src/
│   ├── data.py
│   ├── rule_based.py
│   ├── statistical.py
│   ├── neural.py
│   └── evaluate.py
├── scripts/
│   ├── build_notebook.py
│   └── build_report.py
├── assets/
│   ├── figures/
│   └── bilstm_pos.pt
├── reports/
│   ├── POS_Tagging_Report.pdf
│   ├── output_screenshots.pdf
│   ├── model_comparison.csv
│   └── results.json
├── data/README.md
├── requirements.txt
├── .gitignore
└── README.md
```

Appendix B. Reproduction

```
python -m pip install -r requirements.txt
python -m jupyter nbconvert --to notebook --execute --inplace \
    notebooks/POS_Tagging_Comparison.ipynb
python scripts/build_report.py
```

The notebook downloads the corpus on first execution, trains all five models, saves the comparison artefacts, and regenerates every figure used in this report.

Appendix C. Deliverables Mapping

Deliverable	Location
Complete Jupyter notebook	`notebooks/POS_Tagging_Comparison.ipynb`
Project report (PDF)	`reports/POS_Tagging_Report.pdf`
Output screenshots (PDF)	`reports/output_screenshots.pdf`
Demo video (MP4)	Recorded separately; script in `reports/demo_video_script.md`
GitHub repository	<GitHub Repository URL>